
Capacitated Vehicle Routing Problem

Un algoritmo immunologico a selezione clonale
ibridato con ricerca locale

RELAZIONE DI PROGETTO — HEURISTICS & METAHEURISTICS FOR OPTIMIZATION &
LEARNING

Studente:
Massimiliano Cassia
Matr. 100016487

Docente:
Prof. Mario Pavone

Sommario

Questo lavoro presenta un algoritmo immunologico ispirato al principio della selezione clonale (opzione 2 della consegna), ibridato con una ricerca locale a vicinati multipli, per il *Capacitated Vehicle Routing Problem*. L'elemento portante è la rappresentazione di ogni soluzione come singola permutazione di clienti (*giant tour*), con la suddivisione in percorsi delegata a un decodificatore *esatto* basato su programmazione dinamica (*Split*). Sulle dieci istanze CVRPLIB richieste, con 5 esecuzioni indipendenti ciascuna e criterio di arresto $FE = 3.5 \times 10^5$, l'algoritmo raggiunge uno scarto medio dall'ottimo noto dell'1,82% (minimo 0,17%, massimo 5,58%); tutte le città sono servite in tutte le esecuzioni e il vincolo sul numero di veicoli è sempre rispettato. Un'analisi conclusiva scompone in modo esatto lo scarto residuo, mostrando che esso è interamente imputabile all'*assegnamento* dei clienti ai veicoli e non all'*ordinamento* interno dei percorsi, che risulta ottimo in tutte le soluzioni prodotte.

Indice

1	Introduzione	2
2	Il problema	2
3	Rappresentazione e decodifica esatta (Split)	2
3.1	La scelta della rappresentazione	2
3.2	Il decodificatore Split	3
4	L'algoritmo immunologico a selezione clonale	4
4.1	Motivazione della scelta	4
4.2	Struttura di un anticorpo e ciclo generazionale	4
4.3	Ipermutazione inversamente proporzionale alla fitness	4
4.4	Aging elitista e selezione	5
5	Ricerca locale e ibridazione	5
5.1	Complessità computazionale	6
6	Difficoltà affrontate e scelte progettuali	6
6.1	Convergenza prematura: la soppressione dei duplicati	6
6.2	Il vincolo di flotta e le istanze a capacità satura	7
6.3	L'ipermutazione troppo aggressiva	7
7	Taratura dei parametri	7
8	Originalità introdotte	8
9	Risultati sperimentali	8
10	Analisi critica	9
10.1	Curve di convergenza	9
10.2	Che cosa fanno realmente gli operatori	10
10.3	Da dove viene lo scarto residuo? Una scomposizione esatta	10
10.4	Un esperimento naturale: E-n101-k14 contro P-n101-k4	11
10.5	Opinioni personali sui risultati	12
11	Conclusioni	12

1 Introduzione

Il *Vehicle Routing Problem* (VRP) è uno dei problemi classici dell’ottimizzazione combinatoria: dato un insieme di veicoli e un insieme di clienti geograficamente distribuiti, si devono progettare i percorsi di costo minimo che servono tutti i clienti partendo da un deposito centrale e ritornandovi. Nella variante *capacitata* (CVRP), oggetto di questo progetto, ogni veicolo ha una capacità di carico limitata e la somma delle domande dei clienti serviti da un percorso non può superarla.

Il CVRP è **NP-hard**: contiene come caso particolare il *Travelling Salesman Problem* (basta porre la capacità pari alla domanda totale, ottenendo un unico percorso che visita tutti i clienti). Non è quindi ragionevole attendersi un algoritmo esatto efficiente per istanze di dimensione interessante, e il ricorso a una metaeuristica è pienamente giustificato: rinunciamo alla garanzia di ottimalità in cambio di soluzioni di alta qualità in tempi accettabili.

Contributi del lavoro. Il progetto sviluppa: una **rappresentazione a giant tour** con **decodifica esatta** tramite programmazione dinamica (Sez. 3), che garantisce l’ammissibilità per costruzione e disaccoppia i due sotto-problemi del CVRP; un **motore immunologico a selezione clonale** con ipermutazione inversamente proporzionale alla *fitness*, *aging* elitista e selezione $(\mu + \lambda)$ con soppressione dei duplicati (Sez. 4); una **ricerca locale a cinque vicinati** integrata in schema *lamarckiano* con tetto di valutazioni per invocazione (Sez. 5); una **gestione del vincolo di flotta** per decodifica vincolata e selezione *feasibility-first*, imposta dalle istanze a capacità satura (Sez. 6); e un’**analisi critica quantitativa** che scompone lo scarto residuo e ne individua la causa (Sez. 10).

2 Il problema

Sia $G = (V, E)$ un grafo completo con $V = \{v_0, v_1, \dots, v_n\}$, dove v_0 è il *depot* e $V \setminus \{v_0\}$ è l’insieme dei clienti. A ogni arco (i, j) è associato un costo non negativo d_{ij} . Ogni cliente i ha una domanda $q(i) > 0$; sono disponibili m veicoli, ciascuno di capacità σ .

Una soluzione è un insieme di percorsi $\eta_1, \dots, \eta_m$ tale che ogni cliente sia visitato esattamente una volta da un solo veicolo, ogni percorso inizi e termini al depot e, per ogni veicolo h , valga $\sum_{i \in V_{\eta_h}} q(i) \leq \sigma$. L’obiettivo è minimizzare il costo totale $\sum_{h=1}^m \sum_{(i,j) \in \eta_h} d_{ij}$.

Le istanze e la matrice delle distanze. Le dieci istanze richieste appartengono ai gruppi A, B, E e P della libreria CVRPLIB, in formato TSPLIB con `EDGE_WEIGHT_TYPE = EUCL_2D`. Le distanze sono *intere*, ottenute arrotondando la distanza euclidea:

$$d_{ij} = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0,5 \right\rfloor.$$

La funzione `round` dei linguaggi più diffusi implementa l’arrotondamento *bancario* (il caso 0,5 va al pari), che su alcune coppie di punti differisce da quello TSPLIB e renderebbe i costi non confrontabili con gli ottimi pubblicati. La matrice è stata validata sperimentalmente: per tutte e dieci le istanze, il costo delle soluzioni ottime ufficiali ricalcolato con la nostra matrice coincide *esattamente* con quello dichiarato da CVRPLIB — verifica che fonda ogni confronto quantitativo del seguito.

Una proprietà comune alle dieci istanze. Su tutte e dieci le istanze vale $\lceil \sum_i q(i) / \sigma \rceil = k$, dove k è il numero di veicoli indicato nel nome dell’istanza. Il numero di veicoli disponibili coincide dunque con il *minimo teorico*: la capacità è *satura* (il riempimento medio dei veicoli va dall’87% al 95%). Come si vedrà nella Sezione 6, questa caratteristica — notata durante l’analisi preliminare dei dati e non evidente a priori — ha avuto conseguenze profonde sulla progettazione dell’algoritmo.

3 Rappresentazione e decodifica esatta (Split)

3.1 La scelta della rappresentazione

Il CVRP contiene, intrecciati, due sotto-problemi di natura diversa:

- **l’assegnamento:** *quali* clienti affidare a *quale* veicolo (un problema di partizione con vincolo di capacità);
- **l’ordinamento:** in *quale ordine* visitare i clienti di ciascun veicolo (un TSP su ogni percorso).

La rappresentazione più immediata — una lista di percorsi espliciti — obbliga gli operatori genetici a manipolare simultaneamente entrambi gli aspetti, e soprattutto a preoccuparsi costantemente dell’ammissibilità: qualunque mutazione che sposti un cliente rischia di violare la capacità di un veicolo, imponendo o meccanismi di riparazione o funzioni di penalità.

Abbiamo scelto la strada opposta. Una soluzione è codificata come un **giant tour**: una singola permutazione $S = (s_1, \dots, s_n)$ di tutti i clienti, *priva di separatori di percorso*. La suddivisione in percorsi non è codificata nel genotipo: viene *calcolata in modo ottimo* al momento della valutazione, dal decodificatore *Split*.

3.2 Il decodificatore Split

Fissato l'ordine S , si consideri il grafo ausiliario aciclico H con nodi $0, 1, \dots, n$, in cui l'arco (i, j) con $i < j$ rappresenta la decisione “un veicolo serve i clienti $s_{i+1}, \dots, s_j$ in quest'ordine”. L'arco esiste solo se il carico del segmento non supera la capacità, e il suo costo è

$$c(i, j) = d_{0, s_{i+1}} + \sum_{t=i+1}^{j-1} d_{s_t, s_{t+1}} + d_{s_j, 0}.$$

Poiché ogni cammino da 0 a n in H corrisponde biunivocamente a una suddivisione di S in percorsi ammissibili, il **cammino di costo minimo** da 0 a n fornisce la *migliore* suddivisione possibile del giant tour. H è un DAG già ordinato topologicamente, quindi il cammino minimo si calcola con la ricorrenza di Bellman (Algoritmo 1), un'applicazione diretta della **programmazione dinamica**.

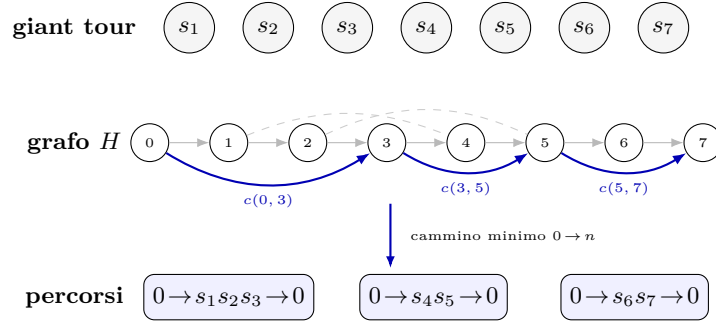


Figura 1: Decodifica Split. Il giant tour (in alto) non contiene separatori. Nel grafo ausiliario H ogni arco rappresenta un possibile percorso (un segmento *contiguo* del tour, ammissibile rispetto alla capacità); tratteggiati alcuni archi alternativi non scelti. Il cammino minimo da 0 a n (in blu) individua la suddivisione ottima, da cui si leggono i percorsi.

Algorithm 1: SPLIT — suddivisione ottima di un giant tour (Bellman su DAG)

Input: giant tour S , domande q , distanze d , capacità σ

Output: costo ottimo $V[n]$ e suddivisione in percorsi

```

1  $V[0] \leftarrow 0$ ;  $V[j] \leftarrow +\infty$ ,
2  $pred[j] \leftarrow -1$  per  $j = 1, \dots, n$ 
3 for  $i \leftarrow 0$  to  $n - 1$  do
4   if  $V[i] = +\infty$  then continue
5    $load \leftarrow 0$ ;
6    $cost \leftarrow 0$ 
7   for  $j \leftarrow i + 1$  to  $n$  do
8      $load \leftarrow load + q(s_j)$ 
9     if  $load > \sigma$  then break // potatura: nessuna estensione è ammissibile
10    if  $j = i + 1$  then
11       $cost \leftarrow d_{0, s_j} + d_{s_j, 0}$  // percorso con un solo cliente
12    else
13       $cost \leftarrow cost - d_{s_{j-1}, 0} + d_{s_{j-1}, s_j} + d_{s_j, 0}$  // estensione in  $O(1)$ 
14    end
15    if  $V[i] + cost < V[j]$  then
16       $V[j] \leftarrow V[i] + cost$ ;  $pred[j] \leftarrow i$ 
17    end
18  end
19 end
20 return  $V[n]$  e i percorsi ottenuti risalendo  $pred$  da  $n$  a 0

```

Complessità. Il ciclo interno si interrompe non appena il carico supera σ : detto s^* il numero massimo di clienti che stanno in un veicolo, il costo è $O(n \cdot s^*)$. L'aggiornamento incrementale di $cost$ in $O(1)$ (riga 9) evita di ricalcolare da capo il costo di ogni segmento. In pratica, su $n = 100$ una decodifica richiede meno di un millisecondo.

Perché questa scelta paga. Quattro vantaggi concreti. *Ammissibilità per costruzione:* ogni soluzione decodificata rispetta la capacità, senza riparazioni né penalità. *Operatori semplici:* le mutazioni agiscono su una permutazione e non possono produrre soluzioni inammissibili. *Ottimalità della suddivisione:* a parità di ordine nessun algoritmo può fare meglio della programmazione dinamica, e la metaeuristica può concentrarsi solo sull'ordinamento. *Paesaggio più regolare:* una singola mutazione può riorganizzare radicalmente i percorsi, perché la decodifica si riadatta.

Verifica di correttezza. Oltre al confronto con un'enumerazione esaustiva su istanze giocattolo, il decodificatore soddisfa una proprietà verificabile in modo esatto: concatenando i percorsi della soluzione *ottima* di CVRPLIB in un giant tour e applicandovi lo Split (vincolato a k percorsi) si riottiene *esattamente* il costo ottimo. La dimostrazione è immediata: la suddivisione ottima è una delle suddivisioni contigue con al più k percorsi, dunque il costo calcolato non può essere superiore all'ottimo; e ogni suddivisione con al più k percorsi è una soluzione ammissibile del CVRP, dunque non può essere inferiore. Questo controllo è superato su tutte e dieci le istanze.

4 L'algoritmo immunologico a selezione clonale

4.1 Motivazione della scelta

Fra le cinque opzioni proposte, la scelta è caduta sull'**algoritmo immunologico a selezione clonale** (opzione 2), ibridato con ricerca locale. Le ragioni sono tre.

In primo luogo, il paradigma immunologico offre un **meccanismo di bilanciamento fra esplorazione e sfruttamento che è intrinseco e adattivo**, e non un parametro fisso: l'ipermutazione modula l'intensità della perturbazione in funzione della qualità dell'individuo, cosicché le soluzioni migliori vengono raffinate con piccoli aggiustamenti mentre quelle peggiori esplorano aggressivamente lo spazio. In un algoritmo genetico classico il tasso di mutazione è invece uniforme sulla popolazione. In secondo luogo, l'*aging* fornisce un meccanismo esplicito e diretto contro la convergenza prematura, problema notoriamente critico nel CVRP, dove il paesaggio di ricerca è ricco di ottimi locali profondi. In terzo luogo, la selezione clonale non richiede un operatore di ricombinazione: il crossover su permutazioni (OX, PMX, ecc.) è sempre un compromesso delicato, e poterne fare a meno riduce le scelte arbitrarie e le fonti di errore.

4.2 Struttura di un anticorpo e ciclo generazionale

Un **anticorpo** è la tripla (giant tour, costo, età). Il costo (la *fitness*, da minimizzare) è ottenuto dalla decodifica della Sezione 3; l'età governa l'invecchiamento. La popolazione ha dimensione B e il ciclo generazionale è riassunto nella Figura 2 e nell'Algoritmo 2.

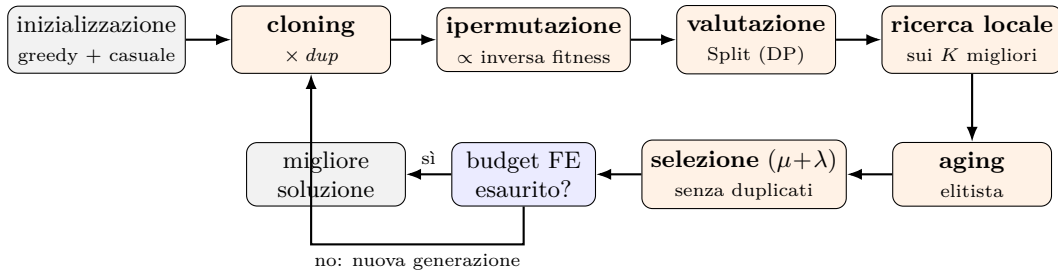


Figura 2: Ciclo generazionale. L'inizializzazione avviene una sola volta; le fasi in arancione sono gli operatori immunologici e l'ibridazione con la ricerca locale, ripetuti a ogni generazione finché il budget di valutazioni non è esaurito.

Inizializzazione: greedy e casuale. Il 25% della popolazione è seminato a partire dall'euristica costruttiva di **Clarke–Wright** (algoritmo dei *savings*, un tipico algoritmo *greedy*): si parte dalla soluzione banale con un veicolo per cliente e si fondono iterativamente i percorsi in ordine decrescente di risparmio $s(i, j) = d_{0i} + d_{0j} - d_{ij}$, quando ammissibile. Il primo individuo è la soluzione di Clarke–Wright pura; gli altri ne sono perturbazioni casuali. Il restante 75% è costituito da permutazioni casuali.

Il bilanciamento è un compromesso: una popolazione interamente greedy sarebbe fatta di individui quasi identici (diversità nulla), una interamente casuale sprecherebbe gran parte del budget per raggiungere una qualità che l'euristica costruttiva fornisce gratuitamente.

4.3 Ipermutazione inversamente proporzionale alla fitness

È l'operatore caratterizzante del paradigma immunologico. Detti c_i il costo dell' i -esimo anticorpo, $c_{\min}$ e $c_{\max}$ il costo minimo e massimo nella popolazione corrente, si definisce la **fitness normalizzata**

$$\hat{f}_i = \frac{c_{\max} - c_i}{c_{\max} - c_{\min}} \in [0, 1], \quad (\hat{f} = 1 \text{ per il migliore, } \hat{f} = 0 \text{ per il peggiore})$$

e il **numero di mutazioni elementari** da applicare al clone:

$$M_i = \max\left(1, \left\lceil n \cdot e^{-\rho \hat{f}_i} \right\rceil\right) \quad \text{con } \rho = \ln n.$$

La scelta $\rho = \ln n$ non è arbitraria: con essa il migliore riceve *esattamente una* mutazione ($n \cdot e^{-\ln n} = 1$) e il peggiore ne riceve n , cioè una randomizzazione pressoché completa. La legge esponenziale realizza dunque, con

un solo parametro, l'intero spettro fra sfruttamento puro e esplorazione pura, assegnando a ciascun individuo l'intensità di perturbazione adatta alla sua qualità. Nel caso degenerare $c_{\max} = c_{\min}$ si pone $\hat{f} = 1$ per tutti; la diversità viene allora reintrodotta dall'aging e dalla soppressione dei duplicati, non da mutazioni più violente.

Ogni mutazione elementare è estratta fra tre operatori: **inversione** di un segmento (peso 0,50), **scambio** di due posizioni (0,25) e **inserimento** di un cliente in altra posizione (0,25). All'inversione è assegnato il peso maggiore perché è la mossa più efficace sulle permutazioni di percorso: preserva tutte le adiacenze interne al segmento, distruggendo solo i due archi agli estremi. Tutte e tre preservano l'invariante di permutazione; l'ammissibilità della soluzione decodificata è poi garantita dallo Split.

4.4 Aging elitista e selezione

Aging. Ogni anticorpo ha un'età, incrementata a ogni generazione. Un clone *eredita* l'età del genitore, ma essa viene **azzerata se il clone migliora il genitore**: le linee evolutive che stanno progredendo vengono così protette, mentre quelle stagnanti invecchiano e muoiono. Superata l'età massima τ_B , l'anticorpo viene rimosso. L'aging è **elitista**: il miglior anticorpo corrente non muore mai, per non perdere l'informazione migliore disponibile.

Selezione ($\mu + \lambda$) con soppressione dei duplicati. Dall'unione di genitori e cloni sopravvissuti all'aging si selezionano i B migliori. La selezione, però, **scarta i duplicati**: a parità di costo passa un solo anticorpo, e i duplicati rientrano solo se non ci sono abbastanza soluzioni distinte. Questa regola, apparentemente minore, si è rivelata essenziale (Sezione 6). Se dopo la selezione la popolazione è sotto organico, si generano **nuovi anticorpi casuali** (nascite), ulteriore iniezione di diversità.

Algorithm 2: Algoritmo immunologico a selezione clonale, ibridato con ricerca locale

```

Input: istanza, budget  $FE_{\max} = 3.5 \times 10^5$ , parametri  $B, dup, \rho, \tau_B, K, cap$ 
Output: migliore soluzione ammissibile trovata
1  $P \leftarrow$  inizializza  $B$  anticorpi (25% da Clarke–Wright, resto casuali); valuta
2 while budget non esaurito do
3    $\hat{f} \leftarrow$  fitness normalizzata di  $P$ 
4    $C \leftarrow \emptyset$ 
5   foreach  $a \in P$  do
6      $M \leftarrow \max(1, \lceil n e^{-\rho \hat{f}_a} \rceil)$  // ipermutazione
7     for 1 to  $dup$  do
8        $c \leftarrow MUTA(a, M)$ ;
9       valuta  $c$  con SPLIT // 1 FE
10       $c.et\grave{a} \leftarrow a.et\grave{a}$ ;  $C \leftarrow C \cup \{c\}$ 
11    end
12  end
13  foreach  $c \in i K$  migliori di  $C$  do
14     $c \leftarrow RICERCALOCALE(c, \text{tetto} = cap \text{ FE})$  // ibridazione lamarckiana
15  end
16  foreach  $c \in C$  che migliora il proprio genitore do  $c.et\grave{a} \leftarrow 0$ 
17  incrementa l'età di  $P \cup C$ ;
18  rimuovi chi ha età  $> \tau_B$  (mai il migliore) // aging
19   $P \leftarrow$  i  $B$  migliori dei sopravvissuti, senza duplicati // selezione ( $\mu + \lambda$ )
20  if  $|P| < B$  then completa con anticorpi casuali (nascite)
21 end

```

5 Ricerca locale e ibridazione

L'ibridazione con la ricerca locale è ciò che porta la qualità delle soluzioni vicino agli ottimi noti: è l'aspetto *hybrid metaheuristics* del progetto. La ricerca locale opera sui *percorsi decodificati* e usa cinque vicinati, esplorati in sequenza con strategia *first-improvement* finché un giro completo non produce miglioramenti (Figura 3): **2-opt** (intra-percorso: inverte un segmento, eliminando gli incroci); **Or-opt** (sposta una catena di 1–3 clienti consecutivi, eventualmente invertendola, nello stesso percorso o in un altro); **relocate** (sposta un cliente in un altro percorso: è il caso di Or-opt con catena unitaria); **swap** (scambia due clienti di percorsi diversi); **2-opt*** (scambia le *code* di due percorsi, e come caso limite ne realizza la *fusione*). I primi due agiscono sull'*ordinamento*, gli ultimi tre sull'*assegnamento*: insieme coprono entrambe le componenti del problema.

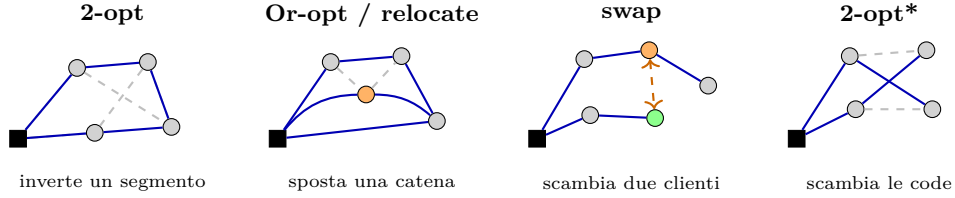


Figura 3: I vicinati della ricerca locale (tratteggiato grigio: archi rimossi; blu: archi creati). I primi due agiscono sull'ordinamento interno ai percorsi, gli ultimi due sull'assegnamento dei clienti ai veicoli.

Valutazione incrementale e liste di candidati. Ogni mossa è valutata in $O(1)$ tramite la variazione di costo Δ (differenza fra archi creati e archi rimossi); i controlli di capacità sono anch'essi in $O(1)$ grazie alle somme prefisse dei carichi. L'esplorazione completa dei vicinati inter-percorso costerebbe $O(n^2)$ valutazioni per passata, insostenibile con un budget di $3,5 \times 10^5$ valutazioni: si adotta quindi la tecnica standard delle **liste di candidati**, considerando per ogni cliente solo mosse che creano archi verso uno dei suoi $h = 20$ vicini più prossimi.

Ibridazione lamarckiana. La soluzione migliorata dalla ricerca locale viene **riscritta nel genotipo**: i percorsi ottimizzati sono riconcatenati in un nuovo giant tour, che sostituisce quello originale dell'anticorpo. Si tratta di apprendimento *lamarckiano* (il carattere acquisito viene ereditato), in contrapposizione allo schema *baldwiniano*, in cui il miglioramento influenzerebbe solo la fitness lasciando intatto il genotipo. La scelta lamarckiana è motivata dalla struttura della rappresentazione: poiché lo Split estrae segmenti contigui, la riconcatenazione dei percorsi ottimizzati è una operazione esatta e senza perdita di informazione, e trasmette ai discendenti il lavoro già svolto dalla ricerca locale.

Il tetto di valutazioni per invocazione. Una discesa completa fino all'ottimo locale costa in media ~ 12000 valutazioni: con l'intero budget se ne potrebbero fare solo 29. Ma il costo non è distribuito uniformemente: gran parte delle valutazioni è consumata dall'ultima passata, quella che scandaglia tutti i vicinati *senza trovare nulla*, per certificare l'ottimalità locale. Abbiamo quindi introdotto un **tetto di FE per invocazione** (*cap*): la discesa si interrompe al tetto, rinunciando alla certificazione ma conservando i miglioramenti ad alto rendimento. Con K , è il parametro più influente dell'algoritmo (Sezione 7).

5.1 Complessità computazionale

Detti n i clienti, B la popolazione, dup i cloni per anticorpo, s^* il massimo numero di clienti in un veicolo, h l'ampiezza delle liste di candidati, una generazione costa: $O(B \cdot dup \cdot n)$ per l'iperpermutazione nel caso peggiore (il peggior anticorpo riceve n mutazioni, ma in media $M_i \ll n$); $O(B \cdot dup \cdot k \cdot n \cdot s^*)$ per la valutazione dei cloni (una decodifica ciascuno, con la DP vincolata a k percorsi); $O(K \cdot cap)$ per la ricerca locale, limitata per costruzione dal tetto, con una passata sui vicinati inter-percorso che costa $O(n \cdot h)$ anziché $O(n^2)$ grazie alle liste di candidati; e $O(B \cdot dup \cdot \log(B \cdot dup))$ per aging e selezione.

Con i parametri congelati il termine dominante è di gran lunga la ricerca locale ($K \cdot cap = 15000$ valutazioni contro le $B \cdot dup = 45$ decodifiche): è questa asimmetria a spiegare perché il budget di $3,5 \times 10^5$ valutazioni si esaurisca in circa 24 generazioni, e perché il tetto *cap* sia il parametro più influente dell'algoritmo. In termini di tempo, una *run* completa richiede fra 1,5 e 6 secondi.

Convenzione di conteggio delle valutazioni. Il criterio di arresto imposto è il numero massimo di valutazioni della funzione di fitness. Adottiamo la convenzione più **conservativa**: una *valutazione* = il calcolo del costo di una soluzione candidata, sia essa una decodifica Split completa oppure la valutazione incrementale Δ di un vicino esaminato dalla ricerca locale. Il fatto che il Δ si calcoli in $O(1)$ è un'ottimizzazione di *velocità*, non una ragione per non contare la valutazione. Una convenzione più permissiva (contare solo le decodifiche complete) consentirebbe una ricerca locale molto più estesa a parità di budget dichiarato; la nostra scelta va nella direzione opposta e non può in alcun caso sovrastimare le prestazioni. L'arresto è esatto: tutte le 50 esecuzioni terminano a 350000 valutazioni, né una in più né una in meno.

6 Difficoltà affrontate e scelte progettuali

Questa sezione documenta i tre problemi che hanno richiesto le decisioni progettuali più delicate. Sono presentati nella forma in cui si sono effettivamente manifestati (sintomo, diagnosi, rimedio).

6.1 Convergenza prematura: la soppressione dei duplicati

Sintomo. Nelle prime esecuzioni complete, l'ultimo miglioramento della soluzione migliore avveniva sistematicamente intorno alla generazione 14, dopo la quale l'algoritmo consumava il restante 75% del budget senza progressi.

Diagnosi. La selezione ($\mu + \lambda$), prendendo semplicemente i B migliori, riempiva la popolazione di *copie* dello stesso ottimo locale. Ma se tutti gli anticorpi hanno lo stesso costo, allora $c_{\max} = c_{\min}$, la fitness normalizzata

degenera ($\hat{f} = 1$ per tutti) e l'ipermutazione si riduce al minimo (una sola mutazione) *per tutti*: l'algoritmo perde esattamente il meccanismo che dovrebbe salvarlo dalla stagnazione. La legge di ipermutazione, da sola, non basta: presuppone diversità nella popolazione.

Rimedio. La *soppressione dei duplicati* in selezione: a parità di costo passa un solo anticorpo. L'effetto è stato immediato e misurabile: su A-n45-k7 lo scarto è sceso dal 2,97% all'1,66%, e soprattutto *l'ultimo miglioramento si è spostato dalla generazione 14 alle generazioni 42–54*, cioè l'algoritmo ha ripreso a progredire fino alla fine del budget.

6.2 Il vincolo di flotta e le istanze a capacità satura

Sintomo. Sull'istanza P-n50-k10 — riempimento medio dei veicoli al 95% — l'algoritmo non produceva *alcuna* soluzione con al più $k = 10$ percorsi.

Diagnosi. Lo Split minimizza il *costo*, non il *numero di percorsi*. Su istanze a capacità satura, la suddivisione a costo minimo di un giant tour usa quasi sempre più di k percorsi: spezzare un percorso in due allunga il costo di poco (due brevi tratte in più verso il depot) ma rilassa enormemente il vincolo di capacità. La misura empirica è impressionante: la percentuale di cloni la cui decodifica *non ammette* una suddivisione in $\leq k$ percorsi arriva al 93% su P-n50-k10 e all'84% su B-n66-k9 (Tabella 3). Il vincolo sul numero di veicoli, che sulle istanze “facili” è automaticamente soddisfatto, qui è il vincolo dominante.

Rimedio, in due passi. Il primo tentativo — decodificare normalmente e poi, solo alla fine, forzare una suddivisione vincolata sulla migliore soluzione — ha prodotto soluzioni conformi ma pessime (16,5% di scarto): la ricerca non stava affatto ottimizzando ciò che poi veniva riportato. Il secondo tentativo — usare direttamente la **decodifica vincolata** a k percorsi come funzione di fitness (una programmazione dinamica bidimensionale $V[r][j]$, con r = numero di percorsi usati) — ha peggiorato ulteriormente (34%): i costi della decodifica *rilassata* (senza vincolo sul numero di percorsi), essendo per costruzione più bassi, dominavano in selezione, e gli anticorpi conformi venivano sistematicamente eliminati.

La soluzione corretta richiede *entrambi* gli ingredienti: decodifica vincolata come fitness e selezione **lessico-grafica feasibility-first**, in cui un anticorpo conforme (al più k percorsi) precede *sempre* un anticorpo non conforme, e solo a parità di conformità decide il costo. Con questa regola di dominanza lo scarto su P-n50-k10 è crollato dal 34% allo 0,57%. Si noti che non si tratta di un approccio a *penalty function* (opzione 5 della consegna): non c'è alcun coefficiente di penalità da calibrare, né una esplorazione controllata della regione non ammissibile; è una semplice regola di dominanza in selezione, che mantiene le soluzioni non conformi nella popolazione come materiale genetico ma impedisce loro di prevalere.

6.3 L'ipermutazione troppo aggressiva

Un tentativo di aumentare l'esplorazione dimezzando ρ (a $\ln(n)/2$, così che anche i migliori ricevano più mutazioni) ha prodotto un risultato paradossale: la soluzione migliore riportata restava bloccata alla *prima* valutazione. La spiegazione è ora evidente alla luce del punto precedente: mutazioni più violente producono tour che si decodificano in più di k percorsi, i quali non possono diventare la soluzione riportata. La lezione è che i parametri di un algoritmo non sono indipendenti dai vincoli del problema: $\rho = \ln n$ non è solo elegante, è anche l'unico valore che mantiene i cloni all'interno della regione ammissibile.

7 Taratura dei parametri

La taratura è stata condotta con una strategia di *coordinate descent* in più round, su un sottoinsieme di quattro istanze eterogenee (una per gruppo: A-n60-k9, B-n66-k9, E-n101-k14, P-n101-k4) e con **tre semi casuali dedicati (100–102), disgiunti dai semi 0–4 usati per gli esperimenti finali**. Questa separazione è una precauzione metodologica essenziale: tarare i parametri sugli stessi campioni casuali con cui poi si misurano le prestazioni significherebbe misurare sui dati di addestramento, sovrastimando la qualità.

Tabella 1: Taratura dei parametri: scarto percentuale medio dall'ottimo sulle 4 istanze di taratura $\times 3$ semi. *Round 1:* forma del budget di ricerca locale (K cloni, tetto *cap* di FE per invocazione). *Round 2:* struttura della popolazione e aging. *Round 3:* ri-validazione delle configurazioni di frontiera sul motore definitivo.

Round 1 — budget LS		Round 2 — popolazione, aging		Round 3 — ri-validazione	
config.	scarto	config.	scarto	config.	scarto
$K=3$, <i>cap</i> =5000	3,11%	$dup=3$	3,00%	$K=3$, <i>cap</i> =5000, $dup=3$, $\tau_B=20$	2,90%
$K=2$, <i>cap</i> =5000	3,26%	$B=10$	3,10%	$K=2$, <i>cap</i> =5000, $dup=3$, $\tau_B=20$	3,00%
$K=3$, <i>cap</i> =3000	4,01%	$B=20$	3,19%	$K=3$, <i>cap</i> =5000, $dup=3$, $\tau_B=10$	3,27%
$K=3$, <i>cap</i> =1500	5,81%	$\tau_B=10$	3,24%	$K=2$, <i>cap</i> =5000, $dup=3$, $\tau_B=10$	3,40%
$K=3$, <i>cap</i> =2000	7,07%	$\tau_B=30$	3,26%		
$K=4$, <i>cap</i> =1500	7,81%	base	3,26%		

I risultati (Tabella 1) indicano con chiarezza che **poche discese profonde battono molte discese superficiali**: a parità di budget di ricerca locale per generazione, le configurazioni con tetto elevato (5000 FE) dominano nettamente quelle con tetto basso. L'interpretazione è che una discesa troncata troppo presto non riesce a uscire dai vicinati più economici (2-opt e Or-opt) e non arriva mai a esplorare le mosse inter-percorso, che sono quelle che riorganizzano davvero la soluzione. Il Round 2 mostra poi che aumentare il numero di cloni per genitore ($dup = 3$) intensifica l'esplorazione attorno alle soluzioni élite, con il beneficio maggiore proprio sull'istanza più ostica (P-n101-k4).

Una nota metodologica onesta. I Round 1 e 2 hanno preceduto le modifiche della Sezione 6 (decodifica vincolata e selezione feasibility-first). Poiché quelle modifiche cambiano l'algoritmo in modo sostanziale, la taratura precedente non era più necessariamente valida: si è quindi eseguito un **round di ri-validazione** sul motore definitivo, sulle quattro configurazioni di frontiera. Ne è emerso un fatto istruttivo: $\tau_B = 10$, che sul motore intermedio era vantaggioso, sul definitivo peggiora sistematicamente — con la selezione feasibility-first le linee conformi impiegano più generazioni a maturare, e un aging aggressivo le elimina prematuramente.

Configurazione finale (congelata). $B = 15$ (popolazione), $dup = 3$ (cloni per anticorpo), $\rho = \ln n$ (ipermutazione), $\tau_B = 20$ (età massima), $K = 3$ (cloni sottoposti a ricerca locale), $cap = 5000$ (tetto di FE per discesa), $h = 20$ (ampiezza liste di candidati), 25% di semina greedy.

8 Originalità introdotte

Riassumiamo qui gli elementi che riteniamo costituiscano il contributo originale del progetto, al di là dell'applicazione dello schema standard di selezione clonale.

1. **Decodifica esatta dentro una metaeuristica.** L'uso di un algoritmo *esatto* (programmazione dinamica) come funzione di valutazione di un algoritmo *approssimato*: la parte del problema che sappiamo risolvere esattamente (la suddivisione, dato l'ordine) viene risolta esattamente; alla metaeuristica resta solo la parte che richiede ricerca.
2. **Gestione del vincolo di flotta per dominanza.** Decodifica vincolata e selezione lessicografica *feasibility-first* sono, per quanto ne sappiamo, una soluzione non standard al problema del numero di veicoli nella rappresentazione a giant tour, ed evitano del tutto la calibrazione — notoriamente fragile — di coefficienti di penalità.
3. **Aging con azzeramento dell'età sui miglioramenti.** Protegge le linee evolutive produttive invece di invecchiarle meccanicamente: l'aging diventa un ricambio dei *fallimenti* più che un ricambio anagrafico.
4. **Soppressione dei duplicati come pre-requisito dell'ipermutazione.** Il legame fra collasso della popolazione e degenerazione della legge di ipermutazione (Sez. 6) non è un accorgimento implementativo: è la constatazione che, in un algoritmo immunologico, mantenere la *varianza* della fitness è condizione necessaria perché l'operatore caratterizzante del paradigma funzioni.
5. **Ricerca locale a budget limitato per invocazione.** Il tetto di FE per discesa — motivato dall'osservazione che la certificazione di ottimalità locale consuma gran parte del costo senza produrre miglioramenti — si è rivelato il parametro più influente dell'intero algoritmo.

9 Risultati sperimentali

Protocollo. Dieci istanze CVRPLIB, 5 esecuzioni indipendenti per istanza (semi 0–4, fissati e dichiarati), criterio di arresto $FE = 3,5 \times 10^5$, parametri congelati dalla taratura. L'intera campagna (50 esecuzioni) richiede circa 150 secondi su un normale portatile. Ogni esecuzione è **riproducibile bit a bit**: a parità di seme il risultato è identico.

Tabella 2: Risultati su 5 esecuzioni indipendenti per istanza ($FE = 3,5 \times 10^5$). *best*: miglior costo fra le 5 esecuzioni; *mean* e *dev. std*: media e deviazione standard campionaria dei migliori costi di ciascuna esecuzione; *iter. medie*: numero medio di generazioni necessarie a raggiungere il migliore risultato di ciascuna esecuzione; *scarto*: distanza percentuale del *best* dall'ottimo noto (BKS). Tutte le città sono servite in tutte le 50 esecuzioni (*satisfability* = 100%) e il numero di percorsi è pari a k .

Istanza	BKS	best	scarto	mean	dev. std	iter. medie	percorsi/ k
A-n45-k7	1146	1148	0,17%	1158,2	9,2	17,4	7/7
A-n60-k9	1354	1358	0,30%	1363,8	4,8	15,4	9/9
A-n80-k10	1763	1811	2,72%	1818,4	8,8	14,8	10/10
B-n56-k7	707	710	0,42%	712,6	1,9	15,6	7/7
B-n66-k9	1316	1332	1,22%	1335,6	3,0	17,0	9/9
B-n78-k10	1221	1239	1,47%	1245,8	5,1	10,8	10/10
E-n76-k8	735	743	1,09%	757,6	8,8	15,2	8/8
E-n101-k14	1067	1109	3,94%	1114,2	4,9	15,0	14/14
P-n50-k10	696	705	1,29%	708,6	3,3	19,2	10/10
P-n101-k4	681	719	5,58%	724,4	4,4	17,8	4/4
<i>scarto medio dei best</i>		1,82%	<i>scarto medio delle medie</i>		2,49%		

Lettura d'insieme. Lo scarto medio dall'ottimo è dell'**1,82%**; sette istanze su dieci restano sotto l'1,5% e tre sotto lo 0,5% (A-n45-k7 si ferma a due unità di costo dall'ottimo). Le deviazioni standard sono contenute (1,9–9,2, cioè meno dell'1% del costo): l'algoritmo è *stabile*, non fortunato. Il numero medio di generazioni necessarie a raggiungere il risultato migliore è compreso fra 10,8 e 19,2 su circa 24 generazioni totali: i miglioramenti continuano cioè fino a ridosso dell'esaurimento del budget, senza stagnazione.

Le due istanze più difficili sono E-n101-k14 (3,94%) e P-n101-k4 (5,58%), entrambe con 100 clienti; segue A-n80-k10 (2,72%). La correlazione con la dimensione è evidente ma non è l'intera storia, come mostra la Sezione 10.

10 Analisi critica

10.1 Curve di convergenza

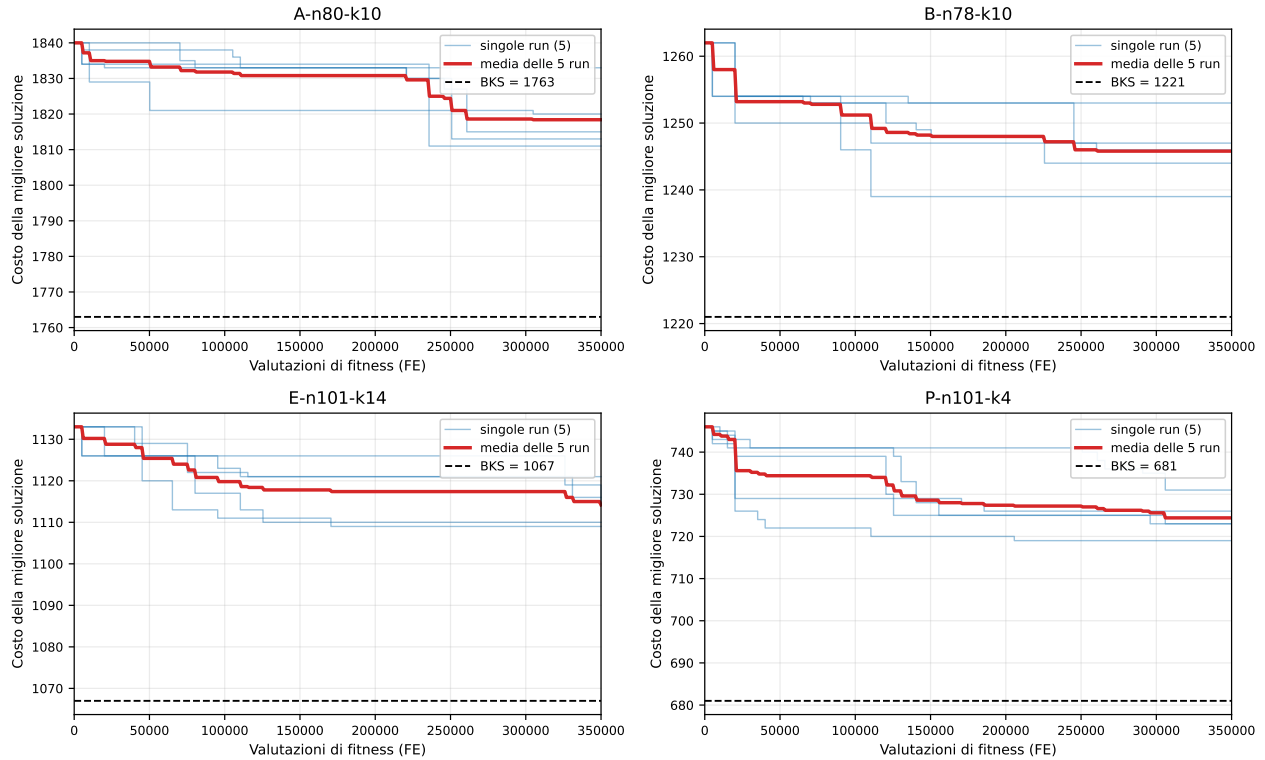


Figura 4: Curve di convergenza su quattro istanze rappresentative (una per gruppo): costo della migliore soluzione trovata in funzione delle valutazioni consumate. In azzurro le 5 esecuzioni, in rosso la loro media, tratteggiato l'ottimo noto.

Le curve (Figura 4) hanno una forma comune: una discesa molto ripida nelle prime 20 000–30 000 valutazioni — dove agiscono l’inizializzazione greedy e le prime discese di ricerca locale — seguita da un lungo miglioramento a gradini, con salti progressivamente più rari ma non nulli. Tre osservazioni critiche.

(a) **Il budget è ben dimensionato ma non sovrabbondante.** Il numero medio di valutazioni necessarie a raggiungere il migliore risultato va da 155 000 (B-n78-k10) a 260 000 (P-n101-k4), su 350 000 disponibili; e confrontando lo scarto a 50 000 valutazioni con quello finale (P-n101-k4: dal 7,84% al 6,37%; E-n101-k14: dal 5,47% al 4,42%) si vede che l’ultima parte del budget contribuisce ancora in modo apprezzabile. *Non* siamo in saturazione: un budget maggiore darebbe verosimilmente risultati migliori — informazione scomoda ma necessaria per interpretare correttamente i numeri.

(b) **La dispersione fra esecuzioni si riduce nel tempo.** Le cinque curve partono separate (domina l’inizializzazione casuale) e convergono in una banda stretta: l’algoritmo è attratto verso una regione di qualità simile indipendentemente dal punto di partenza, coerentemente con le piccole deviazioni standard della Tabella 2.

(c) **Nessuna stagnazione prematura.** Dopo l’introduzione della soppressione dei duplicati non si osservano più i lunghi plateau terminali delle prime versioni: ogni curva migliora anche nell’ultimo quarto del budget.

10.2 Che cosa fanno realmente gli operatori

Una critica legittima a molti lavori su algoritmi bio-ispirati è che gli operatori vengono *descritti* ma non *misurati*: si dà per scontato che funzionino. Abbiamo quindi strumentato l’algoritmo con contatori (che non consumano numeri casuali e non alterano i risultati, verificati identici) per misurare l’attività effettiva di ciascun meccanismo (Tabella 3).

Tabella 3: Attività degli operatori, media su 5 esecuzioni. *Morti aging*: anticorpi rimossi per superamento di τ_B . *Duplicati*: anticorpi declassati dalla soppressione dei duplicati. *LS efficace*: invocazioni della ricerca locale che hanno prodotto un miglioramento, sul totale. *Cloni > k*: percentuale di cloni la cui decodifica non ammette $\leq k$ percorsi.

Istanza	generazioni	morti aging	duplicati	LS efficace	cloni > k
A-n45-k7	24,0	16,6	27,4	70,8 / 71,2	10,2%
A-n60-k9	24,0	24,8	32,2	69,8 / 70,4	40,6%
A-n80-k10	24,0	46,6	18,0	69,0 / 70,2	58,8%
B-n56-k7	24,0	48,8	46,6	69,2 / 70,0	0,2%
B-n66-k9	24,0	6,4	11,6	70,0 / 70,0	83,6%
B-n78-k10	24,0	30,8	25,8	69,2 / 70,0	46,2%
E-n76-k8	24,0	14,4	17,2	70,0 / 70,0	34,8%
E-n101-k14	24,0	26,2	17,8	69,8 / 70,0	46,2%
P-n50-k10	27,2	28,0	36,4	79,6 / 80,6	93,3%
P-n101-k4	24,0	30,4	33,2	69,8 / 70,0	0,0%

I dati confermano che nessun operatore è decorativo. L’**aging** è attivo ovunque (da 6 a 49 rimozioni per esecuzione); il caso di B-n66-k9, dove le morti sono poche, non è un’anomalia ma una conferma del design: lì gli anticorpi migliorano di continuo, l’età viene azzerata, e quindi pochi raggiungono τ_B — esattamente il comportamento voluto. La **soppressione dei duplicati** interviene decine di volte per esecuzione. La **ricerca locale** produce un miglioramento nel $\sim 99\%$ delle invocazioni, il che giustifica il costo in valutazioni che le abbiamo dedicato.

La colonna più eloquente è l’ultima. La percentuale di cloni non conformi al vincolo di flotta varia da 0% a 93%: sono le istanze a capacità satura (P-n50-k10, B-n66-k9) quelle in cui il vincolo sul numero di veicoli è *davvero* il vincolo attivo, e in cui la selezione feasibility-first sta lavorando a pieno regime. Su P-n101-k4 e B-n56-k7, all’estremo opposto, il vincolo è di fatto inattivo. Questa è la misura del perché le scelte descritte nella Sezione 6 erano necessarie e non sovra-ingegnerizzate.

10.3 Da dove viene lo scarto residuo? Una scomposizione esatta

L’analisi più utile che abbiamo condotto risponde a una domanda precisa: lo scarto residuo dall’ottimo è dovuto al fatto che *ordiniamo male* i clienti dentro i percorsi, o al fatto che li *raggruppiamo male* fra i veicoli?

Preso la migliore soluzione trovata, si ri-ottimizza in modo *ottimo* l’ordine di visita all’interno di ciascun percorso, lasciando invariato l’insieme dei clienti assegnati a ogni veicolo (algoritmo di Held–Karp, esatto, applicabile perché i percorsi contengono al più 13 clienti nella quasi totalità dei casi; su P-n101-k4, i cui percorsi arrivano a 30 clienti, si è usata una ricerca locale multi-avvio, calibrata verificando che non riesce a migliorare i percorsi *ottimi* dell’istanza). Si ottiene allora la scomposizione additiva

$$\underbrace{\text{costo trovato} - \text{BKS}}_{\text{scarto totale}} = \underbrace{\text{costo trovato} - \text{costo riordinato}}_{\text{perdita di ordinamento}} + \underbrace{\text{costo riordinato} - \text{BKS}}_{\text{perdita di assegnamento}}.$$

Tabella 4: Scomposizione dello scarto dall’ottimo. La perdita di ordinamento è **nulla su tutte e dieci le istanze**: ogni percorso prodotto dall’algoritmo è già ottimo come TSP.

Istanza	clienti/percorso	costo trovato	costo riordinato	perdita ordin.	perdita assegn.
A-n45-k7	6,3	1148	1148	0	2
A-n60-k9	6,6	1358	1358	0	4
A-n80-k10	7,9	1811	1811	0	48
B-n56-k7	7,9	710	710	0	3
B-n66-k9	7,2	1332	1332	0	16
B-n78-k10	7,7	1239	1239	0	18
E-n76-k8	9,4	743	743	0	8
E-n101-k14	7,1	1109	1109	0	42
P-n50-k10	4,9	705	705	0	9
P-n101-k4	25,0	719	719	0	38
<i>totale</i>				0 (0%)	188 (100%)

Il risultato (Tabella 4) è netto e, va detto, controintuitivo: **la perdita di ordinamento è esattamente zero su tutte e dieci le istanze**. Ogni singolo percorso prodotto dal nostro algoritmo è già ottimo come TSP. Il 100% dello scarto residuo (188 unità di costo in totale) è *perdita di assegnamento*: sbagliamo *quali* clienti mettere insieme, mai *in che ordine* servirli.

Questa scoperta ha una spiegazione strutturale precisa, che riguarda proprio le scelte progettuali descritte sopra. Il vicinato 2-opt intra-percorso è esplorato a *scansione completa* (i percorsi sono corti, quindi ce lo possiamo permettere), mentre i vicinati inter-percorso sono ristretti ai 20 vicini più prossimi; inoltre lo Split può ritagliare solo segmenti *contigui* del giant tour, il che limita strutturalmente i raggruppamenti raggiungibili. La nostra capacità di *ordinare* è quindi molto più forte della nostra capacità di *riassegnare*. È un’informazione preziosa: indica esattamente dove investire per migliorare l’algoritmo, e ci dice che ogni ulteriore sforzo sulla componente TSP sarebbe completamente sprecato.

Un dettaglio istruttivo. Contando gli auto-incroci geometrici dei percorsi (una soluzione 2-opt-ottima, con distanze euclidee reali, non può averne) se ne trova *uno solo* in tutte e dieci le soluzioni, in un percorso di B-n78-k10. Sembrerebbe un difetto della ricerca locale; in realtà quel percorso è *dimostrabilmente* ottimo come TSP (verificato con Held–Karp, costo 84). L’incrocio sopravvive perché le distanze TSPLIB sono *arrotondate all’intero*: la mossa 2-opt che lo eliminerebbe accorcia la lunghezza euclidea reale ma ha $\Delta \geq 0$ sul costo intero. Non è un errore dell’algoritmo, è una proprietà della metrica.

10.4 Un esperimento naturale: E-n101-k14 contro P-n101-k4

Analizzando i dati abbiamo notato che le istanze E-n101-k14 e P-n101-k4 hanno coordinate e domande **identiche**: sono lo stesso insieme di 100 clienti, e differiscono unicamente per la capacità dei veicoli (112 contro 400), quindi per il numero di percorsi (14 contro 4) e per la loro lunghezza (~ 7 contro ~ 25 clienti). È un esperimento naturale controllato che ci viene offerto gratuitamente: l’unica variabile che cambia è la *granularità* della partizione.

I risultati sono 3,94% contro 5,58%: l’istanza a percorsi lunghi è più difficile. Ma — alla luce della scomposizione precedente — *non* perché i percorsi lunghi siano più difficili da ordinare (li ordiniamo comunque in modo ottimo). La ragione è che con 4 percorsi da 25 clienti lo spazio delle partizioni ammissibili è enormemente più vasto e più “piatto”: spostare un cliente da un percorso all’altro cambia pochissimo il costo, i gradienti che guidano la ricerca locale sono deboli, e la decodifica Split — che può produrre solo segmenti contigui del giant tour — fatica a esplorare raggruppamenti radicalmente diversi. La Figura 5 lo mostra visivamente: i nostri quattro percorsi lunghi (b) si intrecciano, mentre l’ottimo (c) li dispone in quattro regioni nettamente separate — eppure entrambe le soluzioni hanno percorsi TSP-ottimi.

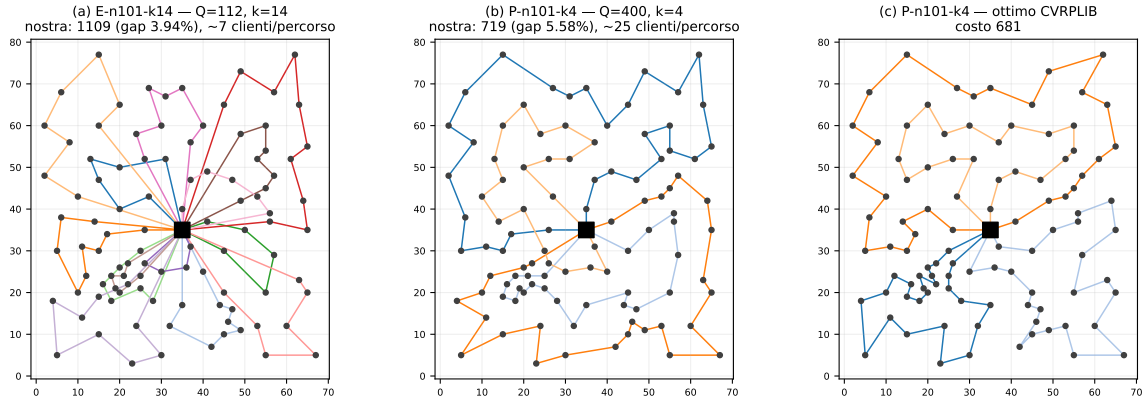


Figura 5: Un esperimento naturale. Le istanze E-n101-k14 (a) e P-n101-k4 (b) condividono *esattamente* la stessa geometria (coordinate e domande identiche) e differiscono solo per la capacità dei veicoli (112 contro 400): 14 percorsi brevi contro 4 percorsi lunghi. In (c) l’ottimo di P-n101-k4: i singoli percorsi sono ottimi come TSP sia in (b) sia in (c); ciò che cambia è la *ripartizione* dei clienti, che nell’ottimo separa il piano in regioni molto più nette.

10.5 Opinioni personali sui risultati

Il risultato che considero più significativo non è lo scarto medio dell’1,82%, ma il fatto che le scelte progettuali più importanti siano emerse dai *fallimenti*, e non dalla teoria. La soppressione dei duplicati, la selezione feasibility-first, il tetto di valutazioni per discesa: nessuna di queste tre era prevista nella progettazione iniziale, e tutte e tre sono state imposte da comportamenti anomali osservati sperimentalmente. In una metaeuristica, l’interazione fra gli operatori genera comportamenti che la descrizione dei singoli operatori non lascia prevedere, e solo la misurazione sistematica li rivela.

Sono soddisfatto della qualità raggiunta, consapevole che essa poggia in larga parte sull’ibridazione: la ricerca locale produce la maggior parte della qualità, mentre il motore immunologico fornisce la diversificazione che le impedisce di restare intrappolata. Un algoritmo immunologico *puro* non sarebbe competitivo; ma neppure una ricerca locale multi-avvio, priva dell’intensificazione attorno alle soluzioni élite, raggiungerebbe questi valori (una singola discesa da Clarke–Wright si ferma fra il 2% e l’8,5% di scarto). Il limite più chiaro resta quello identificato dalla scomposizione: la debolezza nella *riassegnazione* dei clienti fra veicoli — un limite strutturale della rappresentazione a giant tour, il prezzo dell’eleganza della decodifica Split, non un difetto di implementazione.

11 Conclusioni

Il progetto ha prodotto un algoritmo immunologico a selezione clonale ibridato con ricerca locale che raggiunge uno scarto medio dell’1,82% dagli ottimi noti sulle dieci istanze richieste, con esecuzioni interamente riproducibili, tutte le città servite e il vincolo sul numero di veicoli sempre rispettato.

Sul piano tecnico, ritengo che la scelta più felice sia stata quella della rappresentazione a giant tour con decodifica esatta: essa ha reso banale l’ammissibilità rispetto alla capacità, ha semplificato tutti gli operatori e ha permesso di concentrare la ricerca su un unico spazio (le permutazioni). Il suo prezzo — la difficoltà nel riassegnare i clienti fra veicoli — è emerso solo alla fine, dalla scomposizione esatta dello scarto, ed è oggi l’indicazione più precisa su dove intervenire.

Se dovessi proseguire il lavoro, agirei esattamente lì, e non altrove: viciniati inter-percorso più potenti (catene di espulsione, scambi λ -interchange), liste di candidati più ampie, o un operatore di mutazione che agisca esplicitamente sui *gruppi* anziché sull’ordine. Sarebbe inoltre naturale esplorare la variante con *penalty function* (opzione 5 della consegna): il lavoro fatto sul vincolo di flotta mostra che le istanze a capacità satura costringono la ricerca a muoversi lungo il bordo della regione ammissibile, ed è plausibile che consentirne l’attraversamento controllato — accettando temporaneamente soluzioni con più di k percorsi, penalizzate — offrirebbe scorciatoie che oggi ci sono precluse.

Sul piano metodologico, l’esperienza mi lascia una convinzione precisa: la differenza fra un algoritmo che funziona e uno che non funziona non sta quasi mai nella scelta del paradigma, ma nei dettagli di interazione fra gli operatori — dettagli che si scoprono solo misurando, e che nessuna descrizione teorica avrebbe potuto suggerire in anticipo.